

AGRIWISE

SMART CROP ADVISOR & PREDICTOR

PROJECT REPORT

Student Name	Vaidant Udawat
Registration Number	25BAI10266
Batch / Slot	B21+E14
Teacher Name	Dr. Komarasamy G.
Course Name	Programming in JAVA
Project Scope	Crop Recommendation • Cultivation SOP • Profit Analysis

2. TABLE OF CONTENTS

1. Cover Page	1
2. Table of Contents	2
3. Introduction	3
4. Problem Statement	4
5. Functional Requirements	6
6. Non-functional Requirements	7
7. System Architecture	8
8. Design Diagrams	9
9. Design Decisions & Rationale	13
10. Implementation Details	16
11. OOP Mapping	19
12. Screenshots / Results	20
13. Testing Approach	23
14. Challenges Faced	24
15. Learnings & Future Enhancements	26
16. References	28

3. INTRODUCTION

Agricultural decision-making is inherently data-dependent because crop suitability, expected yield, cultivation cost, and market realization vary with geography, season, soil condition, rainfall, temperature, and price movements. Traditional small- and mid-scale farming practice frequently relies on accumulated experience and intuition. While such knowledge remains valuable, intuition alone does not provide a repeatable computational method for comparing alternatives across structured regional datasets. AgriWise is designed as a Core Java command-line application that turns this decision process into an explicit software workflow.

The system models the farmer as a structured profile containing name, location, and land size, and it separates prediction, advisory generation, financial computation, and data representation into dedicated classes. The client-facing entry point, `AgriWiseMain`, collects inputs through `Scanner` and routes the workflow to the business modules. `PredictionEngine` produces a list of candidate `Crop` objects, `AdvisoryGenerator` produces a textual cultivation Standard Operating Procedure (SOP), and `ProfitCalculator` computes expected yield, gross revenue, total cultivation cost, and estimated net profit.

From a software-engineering perspective, the project demonstrates a layered N-Tier organization. The `core` package contains business services, the `data` package contains domain models and CSV resources, and the `tests` package contains the custom verification harness. Collections such as `List<Crop>` provide dynamic in-memory storage of prediction results, while File I/O concepts are represented by the project's CSV datasets and the intended sequential parsing boundary. `BufferedReader` is the appropriate Core Java abstraction for line-oriented CSV ingestion because it supports streaming access without requiring the complete dataset to be materialized in memory.

The project's source package contains three CSV datasets: `historical_weather.csv`, `soil_metrics.csv`, and `mandi_price.csv`. The datasets contain regional weather metrics, soil characteristics, and dated market-price records respectively. An important implementation-fidelity observation is that the supplied `PredictionEngine.java` currently constructs predefined `Crop` objects from season-based conditions rather than reading those CSV files at runtime. Likewise, `ProfitCalculator.java` consumes the price already embedded in the `Crop` object instead of performing a live `mandi_price.csv` lookup. Therefore, the architecture and dataset resources support a data-driven design direction, while the current code represents a compact rule-based prototype of that design.

Project Scope

The project is intentionally constrained to a pure Java CLI environment. Its objective is to demonstrate algorithmic data processing, object-oriented abstraction, flat-file dataset integration, financial computation, and automated verification using standard JDK facilities. No graphical presentation layer or external database dependency is required for the current prototype.

Principal Workflow

1. Capture farmer identity, location, land size, and season through the terminal.
2. Construct a `FarmerProfile` object and pass the location and season to the prediction layer.
3. Generate a candidate crop list and present the recommendations in the CLI.
4. Accept the selected crop and invoke the cultivation-advisory and profit-calculation services.

5. Display step-by-step SOP guidance followed by estimated yield, gross revenue, total cost, and net profit.

4. PROBLEM STATEMENT

Small and mid-scale farmers can face financial uncertainty when crop decisions are made without systematic comparison of environmental and market variables. Weather conditions are not uniform across regions; soil characteristics differ even within similar climatic zones; seasonal suitability changes; and agricultural prices fluctuate over time. When these variables are evaluated mentally or through disconnected information sources, the decision process becomes difficult to reproduce, audit, or scale.

The software-engineering problem is therefore the design of a lightweight advisory system that transforms heterogeneous agricultural inputs into a consistent computational workflow. The system must accept human-entered profile information, map that information to a regional and seasonal prediction rule, represent candidate crops as domain objects, generate actionable cultivation guidance, and quantify financial outcomes for the farmer's specific land size. The design must achieve these goals without relying on a graphical application stack or a heavy server-side database, because the academic objective is to demonstrate Core Java concepts directly.

The supplied project statement explicitly positions the system as a CLI-based intelligent agricultural advisory engine and identifies three local CSV resources as the intended dataset sources. The architecture therefore emphasizes clear abstractions and replaceable modules. Instead of combining all calculations inside one procedural `main()` method, AgriWise delegates prediction, advisory generation, and profit analytics to independent services. This decomposition makes the project easier to reason about, test, and extend.

The project must also address a key consistency challenge: the prediction and profit calculations should remain understandable and deterministic. A small academic system benefits from explicit formulas, visible inputs, and transparent outputs. For example, the current profit calculation follows the sequence `totalYield = expectedYield × acres`, `grossRevenue = totalYield × marketPrice`, `totalCost = cost × acres`, and `netProfit = grossRevenue - totalCost`. This makes the financial result traceable to the selected crop's domain attributes and the farmer's declared land size.

Scope Boundary

The current source code does not implement a transactional marketplace, user authentication, cloud ingestion, or runtime database service. The title includes the broader phrase “Marketplace System,” but the supplied implementation is centered on crop recommendation, cultivation advisory, and profit analysis. The report therefore documents those implemented capabilities and treats deeper marketplace transaction capabilities as outside the demonstrated scope.

5. FUNCTIONAL REQUIREMENTS

The functional requirements define the observable services that the AgriWise application is expected to provide to an operator through its command-line workflow.

- **FR-01 — Farmer Profile Intake & Location Mapping:** Accept farmer name, geographic location, land size in acres, and agricultural season through the terminal. Encapsulate these values in a `'FarmerProfile'` object so that downstream services operate on a structured domain representation.
- **FR-02 — Data-Driven Crop Prediction:** Produce a season- and location-oriented set of crop recommendations. The project package provides `'historical_weather.csv'`, `'soil_metrics.csv'`, and `'mandi_price.csv'` as dataset resources intended to support this decision layer. The current `'PredictionEngine'` implementation demonstrates the rule-routing interface but uses predefined crop objects rather than runtime CSV parsing.
- **FR-03 — Cultivation Advisory Generation:** Generate a concise, step-by-step Standard Operating Procedure for the selected crop. The current `'AdvisoryGenerator'` outputs soil preparation, seed-rate guidance, fertilizer ratio, irrigation guidance, and harvesting guidance in a structured terminal format.
- **FR-04 — Profit Analytics & Market Price Calculation:** Compute estimated total yield, gross revenue, cultivation cost, and net profit for the selected crop and the farmer's declared land area. The calculation is transparent and formula-driven.
- **FR-05 — Candidate Crop Selection:** Present the predicted crops as a numbered list and allow the operator to select one crop for detailed advisory and financial analysis.
- **FR-06 — Graceful Invalid Selection Handling:** Detect a crop-selection value outside the available recommendation range and terminate the workflow without attempting an invalid list access.

6. NON-FUNCTIONAL REQUIREMENTS

The non-functional requirements describe the quality attributes and implementation constraints that shape the architecture. They are important because a recommendation algorithm that produces the correct logical result may still be unsuitable if it is unnecessarily memory-intensive, tightly coupled, difficult to inspect, or fragile during user interaction.

- **NFR-01 — Algorithmic Efficiency:** Sequential CSV scanning should be understood as $O(N)$ with respect to the number of records examined. In-memory recommendation-list access is $O(1)$ by index, while financial calculations are $O(1)$ for a selected `'Crop'`. The design avoids repeated full-dataset transformations within a single simple user flow.
- **NFR-02 — Data Durability:** Dataset information is maintained in flat CSV resources so the prototype can operate without an external relational database server. This reduces deployment complexity and keeps the academic project self-contained.
- **NFR-03 — Memory Efficiency:** Buffered line-oriented file access, represented by the intended `'BufferedReader'` approach, permits sequential processing without loading an entire potentially large CSV file into RAM.
- **NFR-04 — User Interface Constraint:** The system is strictly command-line based. Terminal headings, prompts, separators, and optional ANSI styling are used to create a readable operator workflow without introducing a separate graphical framework.
- **NFR-05 — Maintainability:** Business responsibilities are separated into `'PredictionEngine'`, `'AdvisoryGenerator'`, and `'ProfitCalculator'`, while domain state resides in `'Crop'` and `'FarmerProfile'`. This reduces the amount of logic owned by the `'main()'` method.
- **NFR-06 — Deterministic Verification:** Core behavior should be verifiable using the custom `'AgriWiseTest.java'` harness without requiring JUnit, Maven, or other external testing dependencies.

7. SYSTEM ARCHITECTURE

AgriWise uses a modular N-Tier interpretation based on package boundaries and responsibility separation. The architecture is intentionally lightweight: the client tier owns command-line interaction, the business-logic tier performs prediction, advisory generation, and financial computation, and the persistence/data tier contains domain classes and CSV resources. This design is appropriate for a Core Java academic system because it demonstrates separation of concerns without introducing a distributed deployment model.

Client Tier — AgriWiseMain

`AgriWiseMain.java` is the entry point. It creates the `Scanner`, instantiates each business service, displays the title banner, collects farmer inputs, invokes the prediction engine, presents crop options, and routes the selected crop to the advisory and financial modules. It does not contain the prediction formula or profit calculation; those responsibilities remain in dedicated classes.

Business Logic Tier — PredictionEngine, AdvisoryGenerator, ProfitCalculator

The business tier contains independent services with narrow responsibilities. `PredictionEngine` returns a `List<Crop>` based on location and season. `AdvisoryGenerator` converts a selected crop into step-by-step textual cultivation guidance. `ProfitCalculator` applies deterministic arithmetic over crop attributes and acreage to derive financial metrics. This separation allows each concern to be studied, tested, and replaced independently.

Persistence & Data Tier — CSV Datasets and Domain Models

The data tier contains `Crop.java`, `FarmerProfile.java`, and three CSV resources: `historical_weather.csv`, `soil_metrics.csv`, and `mandi_price.csv`. The weather file contains region-season rainfall and temperature values; the soil file contains regional soil type and nutrient measurements; and the mandi file contains crop, market location, dated price-per-quintal entries. In the current source, these files are packaged resources but are not yet consumed by `PredictionEngine` or `ProfitCalculator` at runtime. They therefore represent the dataset boundary and planned ingestion path rather than a hidden claim of implemented parsing.

Architecture Interaction Summary

Tier	Primary Components	Responsibility	Data Flow
Client	<code>AgriWiseMain.java</code>	Input collection, menu flow, output routing	Farmer input → services
Business Logic	<code>PredictionEngine.java</code> ; <code>AdvisoryGenerator.java</code> ; <code>ProfitCalculator.java</code>	Prediction, SOP generation, financial calculation	Profile/crop values → results
Data / Domain	<code>Crop.java</code> ;	Encapsulated application	Objects shared across layers

	FarmerProfile.java	state	
Dataset / Persistence	historical_weather.csv; soil_metrics.csv; mandi_price.csv	Regional and market reference data	Files → intended parsing boundary

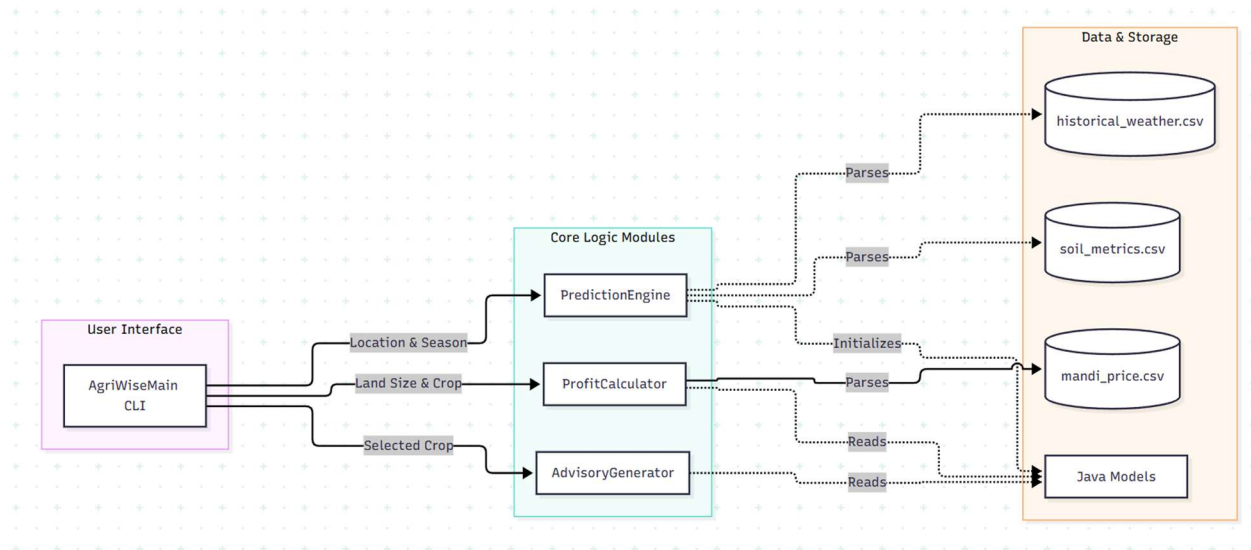


Figure 7.1: N-Tier architecture showing the CLI client, core logic modules, and data/storage resources.

8. DESIGN DIAGRAMS

The following diagrams provide visual representations of the system's behavior and structure. Each figure is inserted directly from the project assets supplied with the report materials. The diagrams are complementary: the use-case view describes externally visible services, the workflow view describes processing stages, the sequence view describes invocation order, the class view describes object relationships, and the ER view represents the conceptual data entities available in the project.

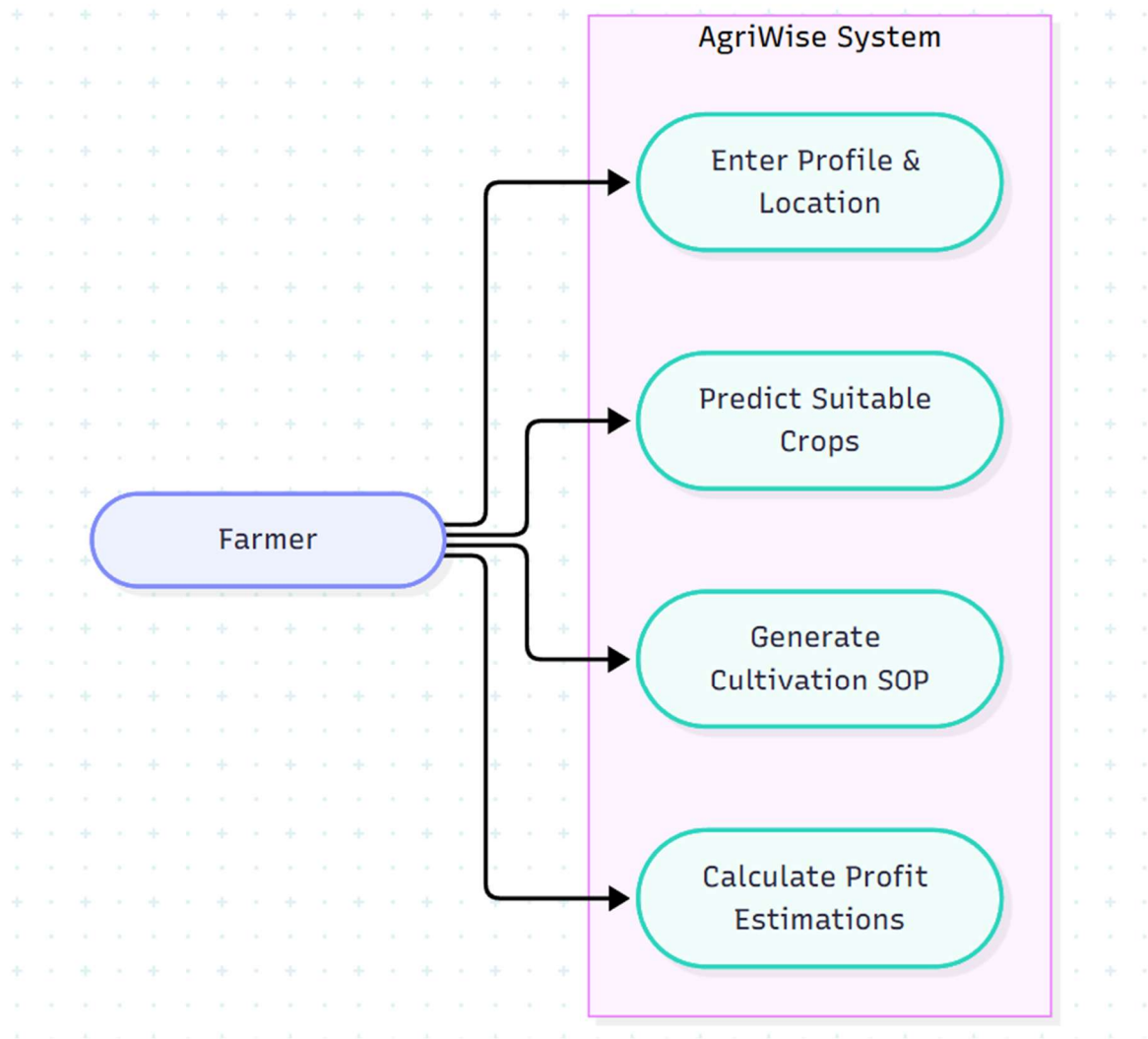


Figure 8.1: Use Case Diagram illustrating Farmer interactions with profile intake, crop prediction, advisory generation, and profit estimation.

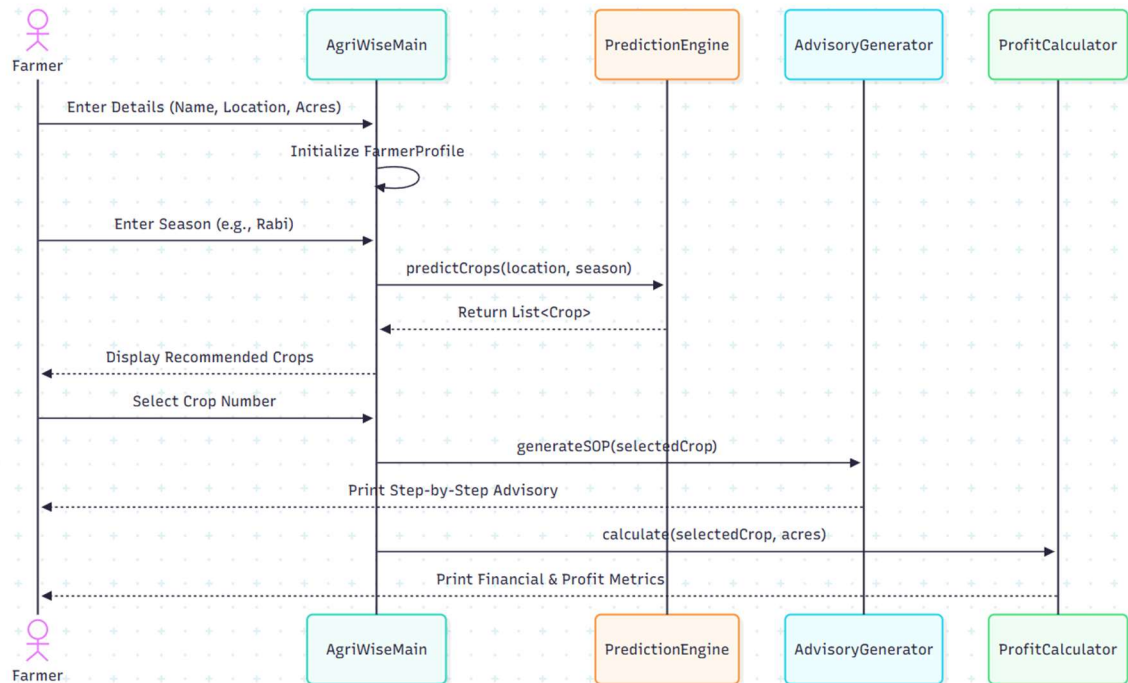


Figure 8.2: Sequence Diagram detailing the chronological method invocations from AgriWiseMain to PredictionEngine, AdvisoryGenerator, and ProfitCalculator.

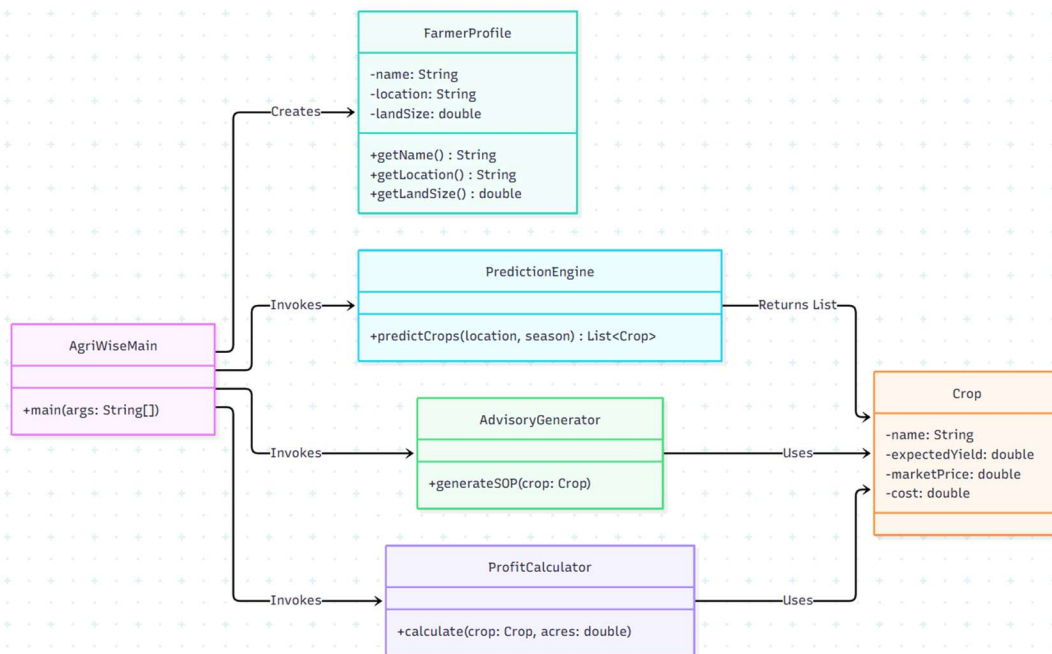


Figure 8.3: Class Diagram showing FarmerProfile, Crop, core business services, and their method-level relationships.

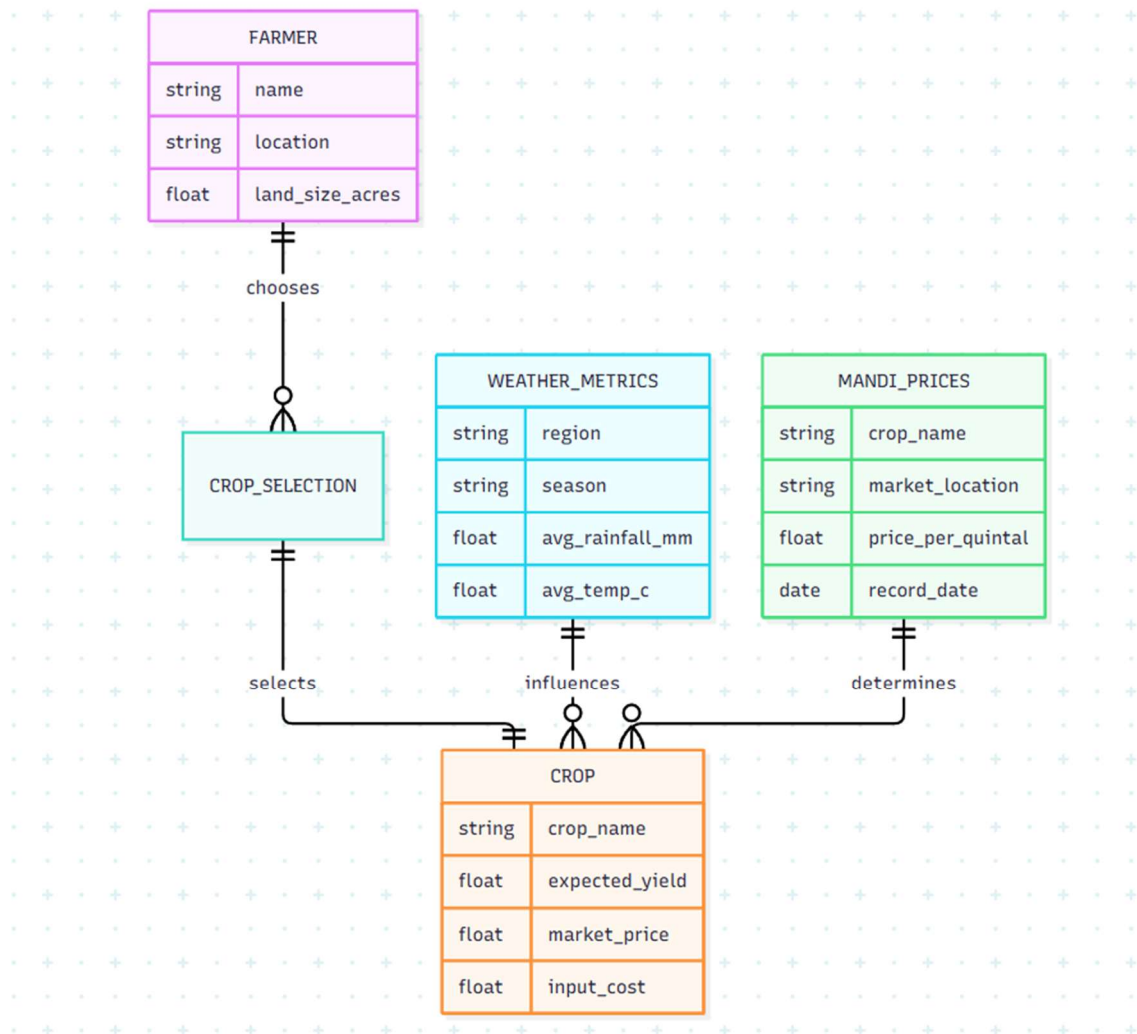


Figure 8.4: Entity-Relationship Diagram representing Farmer, Crop Selection, Weather Metrics, Mandi Prices, and crop-related attributes.

9. DESIGN DECISIONS & RATIONALE

9.1 BufferedReader and Sequential Dataset Processing

For CSV-style resources, `BufferedReader` is a natural Core Java choice because it exposes `readLine()` and supports streaming access over text files. When a dataset contains N records, a one-pass scan is $O(N)$ in time while requiring only bounded working memory for the current line and the derived object being constructed. This is preferable to a design that reads the entire file into a single in-memory string or large intermediate structure when the dataset can grow.

The trade-off is that sequential scanning does not provide indexed lookup. If a farmer repeatedly queries a specific region or crop, the program may revisit records that have already been examined. For an academic prototype, the transparency of sequential parsing is useful; for large-scale deployment, indexed database access would usually be more suitable.

9.2 Lists and Collections

The prediction interface returns `List<Crop>`, allowing the number of recommendations to vary naturally. The list supports indexed retrieval when the operator selects a numbered crop. This is a simpler abstraction than exposing arrays because the collection size can change without manual allocation management.

9.3 Domain Encapsulation

`Crop` and `FarmerProfile` keep their fields private and expose getter methods. This encapsulation prevents external code from modifying internal state directly and gives the classes control over how values are represented. Constructors establish valid object state at creation time. The approach is consistent with object-oriented modeling and keeps domain information distinct from service logic.

9.4 Separation of Calculation Responsibilities

Crop suitability, cultivation procedure generation, and profit computation are separate operations and are therefore modeled by separate service classes. This follows the single-responsibility principle at a practical scale: `PredictionEngine` predicts, `AdvisoryGenerator` explains how the crop should be cultivated, and `ProfitCalculator` performs the financial computation. The resulting architecture is easier to test and maintain than a monolithic controller.

9.5 Transparent Financial Formula

The profit calculation is intentionally deterministic. For acreage A , expected yield Y per acre, market price P per quintal, and cost C per acre: $\text{Total Yield} = Y \times A$; $\text{Gross Revenue} = \text{Total Yield} \times P$; $\text{Total Cost} = C \times A$; $\text{Net Profit} = \text{Gross Revenue} - \text{Total Cost}$. The formulas are simple enough to audit manually, which is valuable in an academic report because the output can be traced directly to input values.

9.6 Dataset Boundary and Implementation Fidelity

The design documents CSV ingestion because the project contains three dataset resources and the project statement identifies them as the intended source of weather, soil, and market information. However, source-level inspection shows that the current `PredictionEngine` returns hard-coded season-based crop objects and that `ProfitCalculator` uses the selected object's stored price. The correct interpretation is that the project establishes the domain abstractions and dataset boundary, while full runtime CSV-driven recommendation and price lookup remain an implementation extension. Documenting this distinction prevents the report from overstating the current software behavior.

10. IMPLEMENTATION DETAILS

10.1 AgriWiseMain.java — CLI Execution & Routing

`AgriWiseMain` initializes the application services and owns the interactive workflow. It creates `Scanner`, `PredictionEngine`, `AdvisoryGenerator`, and `ProfitCalculator` objects, prints the application banner, reads farmer identity and land details, constructs `FarmerProfile`, requests crop predictions, and presents the recommendation list. After the crop is selected, it delegates detailed work to

the advisory and financial services. This structure keeps presentation and routing concerns in one location while preventing business calculations from being embedded inside the entry-point method.

10.2 PredictionEngine.java — Core Prediction Logic

PredictionEngine exposes `predictCrops(String location, String season)` and returns `List<Crop>`. The current implementation branches on season using case-insensitive comparison and adds predefined Crop instances for Kharif, Rabi, or the fallback seasonal path. The method signature provides a clear seam for future CSV-based matching: the location and season parameters already represent the key contextual inputs.

10.3 AdvisoryGenerator.java — Cultivation SOP

AdvisoryGenerator exposes `generateSOP(Crop crop)`. It consumes a domain object instead of raw CLI values, which means the service is decoupled from Scanner and can be invoked by another front end or test harness in a later version. Its output is deliberately structured as numbered instructions covering soil preparation, seed rate, fertilizer, irrigation, and harvesting.

10.4 ProfitCalculator.java — Financial Business Logic

ProfitCalculator receives a selected Crop and the farmer's acreage. It performs four sequential calculations: total yield, gross revenue, total cost, and net profit. The arithmetic is $O(1)$ because a constant number of numeric operations are performed. The result is printed directly to the CLI in a transparent, human-readable form.

10.5 Crop.java — Domain Model & Encapsulation

Crop is a compact domain model with private fields for crop name, expected yield, market price, and cultivation cost. Its constructor initializes state, and getter methods provide controlled access. The class is intentionally free of user-interface logic and can therefore be reused as the shared data contract between prediction and calculation services.

10.6 FarmerProfile.java — Input Domain Model

FarmerProfile encapsulates farmer name, location, and land size. By storing these fields as a domain object immediately after input collection, the system creates a stable representation of the operator context. The model also provides a natural extension point for future attributes such as soil ownership identifiers or additional agricultural constraints.

10.7 CSV Dataset Resources

The supplied dataset files are structured with explicit column headers. `historical_weather.csv` contains Region, Season, Avg_Rainfall_mm, and Avg_Temp_C; `soil_metrics.csv` contains Region, Soil_Type, pH_Level, and N/P/K fields; and `mandi_price.csv` contains Crop, Market_Location, Price_Per_Quintal, and Date. These schemas provide a clear basis for implementing record matching and price lookup in a future version.

10.8 Testing Component — AgriWiseTest.java

AgriWiseTest is a lightweight custom test harness. It instantiates PredictionEngine, requests a Rabi prediction for Punjab, and checks that the returned list is non-empty and that the first recommendation is Wheat. The harness reports a [PASS] or [FAIL] message and can be compiled with the standard JDK.

11. OOP MAPPING

The project applies object-oriented principles through domain modeling, service decomposition, encapsulated state, and interface-based collection usage. The table below maps the principal concepts directly to the supplied source structure.

OOP / Java Concept	Project Location	Concrete Application	Academic Significance
Encapsulation	Crop.java; FarmerProfile.java	Private fields with constructors and getter methods	Protects internal state and exposes controlled access
Abstraction	core service classes	Prediction, advisory, and profit responsibilities exposed through focused public methods	Hides internal calculation details behind service boundaries
Polymorphism / Interface Usage	List<Crop> and standard Java collections	Services depend on collection interfaces rather than concrete iteration semantics	Demonstrates programming to an abstraction
Inheritance	No custom subclass hierarchy in supplied source	The current code does not define a project-specific inheritance tree	The design avoids artificial inheritance where composition/service separation is sufficient
Method Encapsulation	PredictionEngine / AdvisoryGenerator / ProfitCalculator	Each public method represents a focused operation	Reduces coupling between CLI routing and business rules
Object Composition	AgriWiseMain	Creates and coordinates multiple service objects and domain objects	Demonstrates collaboration among independently responsible objects

A strict academic distinction is important here: the supplied implementation clearly demonstrates encapsulation and abstraction, and it uses interface-oriented collection types such as `List<Crop>`. It does not contain a custom inheritance hierarchy. Therefore, inheritance should not be overstated as a major feature of the current source; the strongest OOP evidence is the separation of domain state and business behavior into cohesive classes.

12. SCREENSHOTS / RESULTS

The application was successfully executed in a Java terminal using the supplied project source. The following screenshots are authentic execution evidence from the project materials. They are presented in chronological workflow order: application boot and farmer intake, crop prediction, cultivation advisory,

and financial output. Because the third supplied image contains both SOP and profit sections, it has been separated into two report figures so each reported result remains visually clear.

```
D:\Coding\Java\Java-Vaidant\AgriWise>javac core/*.java data/*.java AgriWiseMain.java

D:\Coding\Java\Java-Vaidant\AgriWise>java AgriWiseMain
=====
AgriWise - Smart Crop Advisor & Predictor
=====
Enter Farmer Name: Rohan
Enter Location (e.g., MP, Punjab): UP
Enter Land Size (in Acres): 67
Enter Season (Kharif / Rabi / Zaid): Kharif
```

Figure 12.1: CLI boot sequence and farmer profile intake showing Java compilation, application initialization, location, land-size, and season capture.

```
=====
AgriWise - Smart Crop Advisor & Predictor
=====
Enter Farmer Name: Rohan
Enter Location (e.g., MP, Punjab): UP
Enter Land Size (in Acres): 67
Enter Season (Kharif / Rabi / Zaid): Kharif

[SYSTEM] Analyzing Data...
Recommended Crops for Kharif:
1. Rice
2. Cotton

Select a crop number for detailed SOP and Profit Analysis: 2
```

Figure 12.2: Crop prediction output showing farmer inputs, analysis status, and recommended Kharif crops.

```
Select a crop number for detailed SOP and Profit Analysis: 2

--- CULTIVATION ADVISORY (SOP) ---
SOP for: Cotton
1. Soil Preparation: Plow 2 times and level.
2. Seed Rate: Optimal spacing required.
3. Fertilizer: Apply NPK in 4:2:1 ratio.
4. Irrigation: Maintain moisture at critical stages.
5. Harvest: Cut when moisture content drops.
```

Figure 12.3: Cultivation advisory output showing the step-by-step SOP generated for the selected Cotton crop.

```
--- PROFIT PREDICTION ---
Crop Selected: Cotton
Total Yield (Quintals): 1005.0
Gross Revenue (INR): 6030000.0
Total Cost (INR): 1340000.0
Net Profit (INR): 4690000.0
```

Figure 12.4: Profit analysis output showing selected crop, estimated yield, gross revenue, cultivation cost, and net profit.

13. TESTING APPROACH

AgriWise includes a custom automated test harness, `tests/AgriWiseTest.java`, designed to verify a core prediction invariant without relying on external frameworks such as JUnit. This choice keeps the project portable: the test can be compiled and executed directly using the standard JDK commands documented in the project README.

Test Methodology

The test harness creates an instance of `PredictionEngine`, invokes `predictCrops("Punjab", "Rabi")`, and then checks two explicit conditions: the returned list contains at least one crop, and the first recommendation is named `Wheat`. The result is printed as `[PASS]` when the condition holds and `[FAIL]` otherwise.

Test ID	Input	Assertion	Expected / Demonstrated Result
T-01	Location = Punjab; Season = Rabi	Prediction list is non-empty	At least one crop is returned
T-02	Same call	First crop name equals Wheat	PASS when Wheat is returned at index 0

This is a focused logic test rather than full end-to-end verification. It does not currently validate CSV parsing, malformed records, invalid numeric input, profit arithmetic, advisory text, or integration between all modules. The scope of the harness should therefore be described accurately: it provides deterministic evidence for a central prediction-path behavior, not complete system-level test coverage.

[INSERT SCREENSHOT: docs/screenshots/ss_5_testing.png]

No testing screenshot was supplied with the current project materials; the bracketed placeholder above is intentionally retained so an authentic execution capture of `java tests.AgriWiseTest` can be inserted before final submission.

14. CHALLENGES FACED

The development of AgriWise involved several practical engineering challenges that are typical of Core Java command-line systems. These challenges are valuable academically because they expose the difference between writing isolated methods and integrating multiple components into a coherent application.

- **Scanner buffer and input-state handling:** Mixing `nextDouble()` or `nextInt()` with subsequent `nextLine()` calls can leave a newline character in the input buffer. The supplied implementation explicitly consumes the pending newline after reading land size before requesting the season. This small detail is essential for reliable CLI interaction.
- **CSV parsing robustness:** Agricultural datasets may contain spacing differences, missing fields, unexpected region names, or additional records. A scalable file-reading layer must parse each line carefully, validate column counts, normalize text comparisons, and handle malformed input without terminating the entire workflow. The current source establishes the dataset boundary, while full runtime parsing remains an extension.
- **Separation of concerns in a compact application:** It is easy for a CLI project to become a single large `main()` method that performs input handling, prediction, calculations, and output. Separating `PredictionEngine`, `AdvisoryGenerator`, `ProfitCalculator`, and the domain models requires deliberate interface design and makes the codebase more maintainable.
- **Consistency between dataset design and runtime logic:** The project materials describe dataset-driven recommendation and market-price analysis, but the supplied implementation currently uses predefined `Crop` objects and their embedded prices. Recognizing and documenting this boundary prevents a misleading report and identifies the precise next step for strengthening the implementation.

15. LEARNINGS & FUTURE ENHANCEMENTS

15.1 Learnings & Key Takeaways

The central learning from AgriWise is that data structures and software abstractions become meaningful when they model a real operational requirement. The `List<Crop>` abstraction is not simply a convenient Java container; it represents a variable set of recommendations that can be generated and presented to the operator. Likewise, buffered text-file access is not merely a syntax exercise: it addresses the practical requirement of processing external datasets without unnecessary memory consumption.

The project also reinforces the value of object-oriented decomposition. `Crop` and `FarmerProfile` keep data separate from business logic, while the three core service classes each represent a focused responsibility. This organization makes the system easier to extend because a future implementation can replace the prediction mechanism or persistence strategy without rewriting every part of the CLI.

A second important learning is the necessity of implementation fidelity. A technical report should distinguish between what the architecture intends to support and what the current source demonstrably executes. In the supplied project, the presence of CSV datasets creates a strong foundation for data-driven behavior, but the present `PredictionEngine` still uses fixed season-based mappings. Recognizing this difference is part of sound engineering evaluation rather than a weakness to be hidden.

15.2 Future Enhancements

JavaFX or Web Presentation Layer: The current CLI can be retained as the interaction contract while a future version adds a graphical front end. The key architectural requirement would be to keep the business services independent of presentation code so the new interface invokes PredictionEngine, AdvisoryGenerator, and ProfitCalculator through stable service boundaries.

PostgreSQL Relational Database: The three CSV resources can be migrated into normalized relational tables for weather metrics, soil metrics, crop master data, and market prices. PostgreSQL would enable indexing, structured queries, transactional updates, and more precise consistency guarantees as the dataset grows.

Runtime CSV / Market Integration: A dedicated ingestion component can be introduced to parse historical_weather.csv, soil_metrics.csv, and mandi_price.csv through BufferedReader, validate records, and populate in-memory collections or repositories. Profit analysis can then query the most recent price record rather than relying on a value embedded directly in the Crop object.

Multi-threaded Data Ingestion: For larger datasets, independent worker tasks could ingest weather, soil, and market resources concurrently before a synchronization barrier combines their results. The benefit would be improved throughput for data preparation, provided the implementation protects shared collections and maintains deterministic merge semantics.

Marketplace Capability: Although the current source does not implement purchase, sale, listing, or transaction processing, a future marketplace module could extend the same domain-oriented architecture with authenticated seller/buyer roles, product listings, transaction records, and persistent order state. Such features would need to be specified separately rather than inferred from the current advisory code.

15.3 Scalability Direction

A scalable version should introduce repository or service interfaces around dataset access, establish validation rules for numeric and categorical inputs, add structured logging, and expand automated tests to cover edge cases such as empty files, malformed rows, missing regions, unexpected seasons, zero or negative acreage, and extreme numeric values. These changes would preserve the core design while strengthening operational reliability.

16. REFERENCES

Oracle. Java SE 17 API Specification — `java.io.BufferedReader`: official documentation for buffered character-input streams and line-oriented reading.

<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/BufferedReader.html>

Oracle. Java SE 17 API Specification — Collections Framework: official documentation covering List and collection interfaces used by Core Java applications.

<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/package-summary.html>

Oracle. Java SE 17 API Specification — `java.util.Scanner`: official documentation for token-based command-line input.

<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Scanner.html>

Oracle. Java SE 17 API Specification — `java.util.ArrayList`: official documentation for resizable-array List implementations.

<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/ArrayList.html>

AgriWise Project Source Package. `Readme.md`, `statement.md`, Java source files, CSV datasets, UML/architecture diagrams, and `AgriWiseTest.java`, supplied as part of the project submission materials.

Academic Implementation Note

This report is aligned to the supplied AgriWise source package, datasets, diagrams, and execution screenshots. Where the requested architectural description is broader than the present source implementation, the distinction is stated explicitly instead of being presented as completed functionality.